راهنمای ساده و کاربردی

ساختار فایل‌ها و روش کار در پروژه‌های AI Agent

از شناخت فایل‌های راهنما تا استفاده از Workflow، Skill و ساخت نمونه‌های واقعی

ارائهٔ آموزشی بر اساس پنج صفحهٔ پروژه

نقشهٔ ارائه

در این ارائه چه می‌آموزیم؟

هدف این است که بدانیم Agent فقط یک چت‌بات نیست؛ بلکه با کمک فایل‌های راهنما یاد می‌گیرد در یک پروژه چگونه تصمیم بگیرد و کار کند.

۱. ساختار فایل‌ها

هر فایل چه نقشی دارد و Agent از کجا شروع می‌کند؟

۲. Workflow و Skill

فرق «مسیر انجام کار» با «توانایی انجام کار» چیست؟

۳. مرور Markdown

فایل‌های مستندات را با چه ترتیبی و برای چه بخوانیم؟

۴ و ۵. دو نمونهٔ واقعی

چطور از دستورالعمل‌ها برای ساخت Case و Roadmap استفاده کنیم؟

ایدهٔ اصلی: فایل‌های Markdown در این پروژه فقط توضیح نیستند؛ بخشی از رفتار، محدودیت و روش تصمیم‌گیری Agent را مشخص می‌کنند.

صفحهٔ ۱: file-structure.html

ساختار فایل‌های AI Agent یعنی چه؟

پروژه را مثل یک ساختمان در نظر بگیرید: قوانین عمومی روی تابلوی ورودی هستند، راهنمای هر متخصص در اتاق مخصوص او قرار دارد و جزئیات فنی در پرونده‌های همان اتاق نگهداری می‌شود.

```
ai-agent-structure-guide/
├── AGENTS.md                    ← قوانین عمومی پروژه
└── .agents/
    └── bmsd-clean-connector/    ← توانایی تخصصی
        ├── SKILL.md             ← نقطهٔ ورود و نقشهٔ کار
        └── references/          ← جزئیات موردنیاز
            ├── rules.md
            ├── architecture.md
            ├── convention.md
            ├── reference.md
            └── security.md
```

قوانین عمومی

در ریشهٔ پروژه و قابل استفاده برای همهٔ کارها.

قابلیت تخصصی

در پوشهٔ **.agents** و مخصوص یک حوزه.

مرجع جزئی

فقط وقتی لازم است خوانده می‌شود.

صفحهٔ ۱: file-structure.html

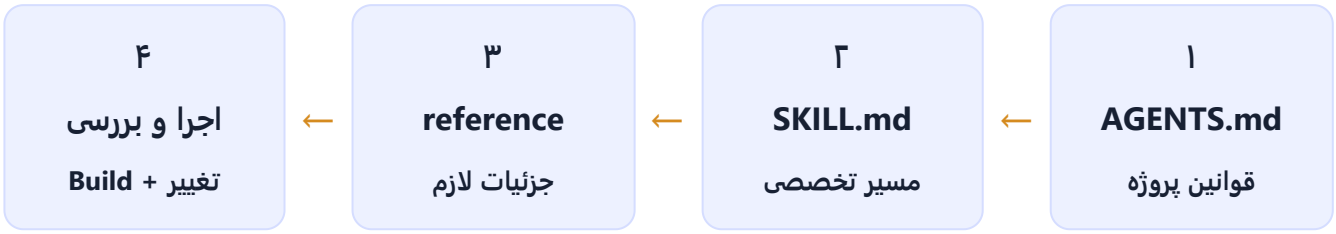
نقش هر فایل به زبان ساده

فایل	نقش	یک مثال
AGENTS.md	قوانین سطح پروژه و مرزهای مجاز تغییر	فقط .NET 8، حفظ Clean Architecture، اجرای Build
SKILL.md	نقطهٔ ورود یک توانایی تخصصی و ترتیب کار	ابتدا پروژه را بررسی کن، سپس مرجع لازم را بخوان
rules.md	قواعد اجرایی و کنترل کیفیت	استفاده از MediatR و لاگ‌نکردن رمزها
architecture.md	مسئولیت لایه‌ها و جهت وابستگی	Domain ← Application ← Infrastructure ← Presentation
convention.md	نام‌گذاری، Route و شکل مدل‌ها	نام صحیح Controller و Command
reference.md / security.md	جزئیات فنی ثابت و نکات امنیتی	تنظیمات CRM، نسخهٔ پکیج‌ها و رمزنگاری

نکته: اسم فایل‌ها قانون جهانی نیست. مهم این است که مسئولیت‌ها واضح باشند و راه رسیدن از راهنما به مرجع درست مشخص باشد.

صفحهٔ ۱: file-structure.html

Agent این فایل‌ها را چگونه استفاده می‌کند؟



مثال

اگر درخواست دربارهٔ ساخت یک Connector باشد، Agent ابتدا قوانین عمومی را می‌بیند. سپس Skill مربوط به Connector را پیدا می‌کند. بعد فقط مثلاً **architecture.md** و **rules.md** را می‌خواند و در پایان پروژهٔ مرتبط را Build و بررسی می‌کند.

فایدهٔ این نظم

- کاهش حدس‌زدن Agent
- جلوگیری از خواندن اطلاعات اضافی
- قابل‌نگهداری‌بودن مستندات

نتیجهٔ نهایی

- کد در جای درست قرار می‌گیرد
- قواعد معماری حفظ می‌شود
- خروجی فقط «کامپایل‌شدن» نیست

صفحهٔ ۳: difference-workflow-skill.html

Workflow و Skill چه تفاوتی دارند؟

Workflow = نقشهٔ انجام کار

مجموعه‌ای از مراحل برای رسیدن به یک هدف. روی ترتیب، شرط و وابستگی مراحل تمرکز دارد.

- شروع، مراحل میانی و پایان دارد
- می‌تواند چند Skill را صدا بزند
- معمولاً یک فرایند کامل را پوشش می‌دهد

Skill = توانایی انجام کار

یک قابلیت تخصصی و قابل استفادهٔ مجدد. روی ورودی، خروجی و روش انجام یک توانایی تمرکز دارد.

- یک کار مشخص را بلد است
- در چند Workflow قابل استفاده است
- معمولاً کوچک‌تر و تخصصی‌تر است

حفظ‌کردن ساده: Workflow می‌پرسد «چه مراحلی باید انجام شوند؟»؛ Skill می‌پرسد «چه کاری را می‌توانم انجام دهم؟»

مثال روزمره: آشپزی

Workflow مثل دستور پخت است: اول مواد را آماده کن، بعد تفت بده و در پایان سرو کن. **Skill** مثل خردکردن، اندازه‌گیری یا کار با فر است؛ هرکدام می‌توانند در دستورهای مختلف دوباره استفاده شوند.

صفحهٔ ۳: difference-workflow-skill.html

یک Workflow چگونه از Skillها استفاده می‌کند؟

فرض کنیم Agent باید گزارش خطای نرم‌افزار را برای تیم فنی آماده و ارسال کند:

1. **هدف:** ساخت گزارش قابل‌فهم و امن
2. **گام اول:** دریافت فایل Log
3. **گام دوم:** تحلیل Log با Skill مربوط
4. **گام سوم:** خلاصه‌سازی خطا
5. **گام چهارم:** حذف اطلاعات حساس
6. **گام پنجم:** ارسال گزارش نهایی، فقط اگر شرایط برقرار باشد

```
Workflow: بررسی و گزارش خطا

1. دریافت فایل لاگ
2. analyze-log
3. summarize-error
4. redact-secrets
5. send-report
```

> **رابطهٔ مهم:** خروجی یک Skill می‌تواند ورودی Skill بعدی باشد؛ Workflow این زنجیره و شرط پایان را مدیریت می‌کند.

موضوع	Workflow	Skill
تمرکز	فرایند و ترتیب	توانایی و قابلیت
استفادهٔ مجدد	اغلب برای یک فرایند مشخص	در چند Workflow
مثال	بررسی و ارسال گزارش	تحلیل Log

صفحهٔ ۳: md-review.html

چرا باید فایل‌های Markdown را مرور کنیم؟

این فایل‌ها فقط برای خواندن انسان نیستند. آن‌ها تصمیم‌های پروژه را ثبت می‌کنند تا Agent و توسعه‌دهنده هر بار از صفر حدس نزنند.

AGENTS.md
قانون کلی

←

SKILL.md
راهنمای توانایی

←

references/*.md
مرجع لازم

←

اجرا + بررسی
خروجی نهایی

اگر مرور نکنیم چه می‌شود؟

- معماری ممکن است شکسته شود
- نام‌ها و Routeها ناهماهنگ می‌شوند
- پکیج یا الگوی اشتباه انتخاب می‌شود
- اطلاعات حساس وارد Log می‌شود

هنگام مرور چه بپرسیم؟

- چه چیزی نوشته شده؟
- چرا نوشته شده؟
- اگر نباشد چه مشکلی رخ می‌دهد؟
- این قانون عمومی است یا تخصصی؟

هر قانون باید در نزدیک‌ترین فایل به مسئولیت خودش قرار بگیرد؛ قانون عمومی در AGENTS و جزئیات تخصصی در references.

صفحهٔ ۳: md-review.html

مرور هر فایل با یک جمله

فایل	چه چیزی می‌گوید؟	چرا مهم است؟
AGENTS.md	هدف پروژه، .NET 8، Clean Architecture و محدودیت تغییر	مرزهای کل پروژه روشن می‌شود.
SKILL.md	این توانایی برای چیست و از چه ترتیبی پیروی کنیم	مسیر شروع و مرجع بعدی را نشان می‌دهد.
rules.md	نحوهٔ اجرا، MediatR، Service و Quality Gate	کد فقط Compile نشود؛ کیفیت و امنیت هم رعایت شود.
architecture.md	مسئولیت Domain، Application، Infrastructure و Presentation	هر منطق در لایهٔ درست قرار گیرد.
convention.md	نام Route، Command، Controller و مدل‌ها	خروجی‌ها یکدست و قابل پیش‌بینی باشند.
reference.md	تنظیمات CRM، Repository و نسخهٔ پکیج‌ها	Agent چیزی را از خودش اختراع نکند.
security.md	رمزنگاری، Credential و ممنوعیت Logکردن Secret	سازگاری و امنیت حفظ شود.

صفحهٔ ۴: sample1.html

نمونهٔ ۱: ساخت Case با شمارهٔ موبایل

در این نمونه، به Agent یک درخواست روشن می‌دهیم تا یک endpoint برای ساخت Case ایجاد کند.

ورودی‌های endpoint

- **caseTitle** از نوع string
- **customerMobilePhone** از نوع string

منطق مورد انتظار

1. Contact را با فیلد **mobilephone** پیدا کن.
2. Contact پیداشده را در فیلد **customer** روی Case قرار بده.
3. Case را با عنوان **caseTitle** ایجاد کن.
4. قوانین معماری و الگوهای پروژه را رعایت کن.

```
یک endpoint بساز با ورودی‌های caseTitle و customerMobilePhone
که هر دو از نوع string باشند.
سپس آن Contact را پیدا کن، Contact را با mobilephone روی customer
قرار بده و Case را با عنوان caseTitle بساز.
در Case قوانین معماری و الگوهای پروژه را رعایت کن.
```

فایل نمونه: Sample1.zip شامل فایل‌های Entity و Resource است و به Agent کمک می‌کند شکل واقعی پروژه را بفهمد.

صفحهٔ ۴: sample1.html

از Prompt تا پیاده‌سازی درست

یک Prompt خوب فقط نمی‌گوید «کد بزن». ورودی، رابطهٔ داده، عملیات و محدودیت معماری را مشخص می‌کند.

چیزهایی که Prompt مشخص کرده

- نام و نوع ورودی‌ها
- نحوهٔ پیدا کردن Contact
- رابطهٔ Contact با Case
- عنوان Case
- رعایت الگوی پروژه

چیزهایی که Agent باید بررسی کند

- Entityهای موجود
- Resource و نام فیلدها
- لایهٔ مناسب برای منطق
- روش خطا در نبود Contact
- Build و تست نهایی

چک‌لیست نتیجه

✓ ورودی‌ها دقیق هستند ✓ رابطهٔ داده درست است ✓ معماری حفظ شده ✓ پروژه Build شده

نکتهٔ عملی: اگر نام فیلدها یا الگوی پروژه را نمی‌دانیم، نباید حدس بزنیم؛ ابتدا فایل‌های راهنما و نمونه را می‌خوانیم.

صفحهٔ ۵: sample2.html

نمونهٔ ۲: ساخت تمپلیت عمومی Roadmap

در نمونهٔ دوم، هدف ساخت یک قالب عمومی برای مدیریت مراحل کار است؛ قالبی که به یک کسب‌وکار یا تعداد ثابت مرحله وابسته نباشد.

هر Phase چه دارد؟

- عنوان و هدف
- توضیح، وابستگی و Scope
- معیار تحویل
- Status و یادداشت تیم
- ترتیب و جزئیات توسعه

قابلیت‌های لازم

- ساخت، ویرایش و حذف Phase
- افزودن و ویرایش Detail
- تغییر ترتیب و فیلتر
- Import و Export
- حفظ اطلاعات بعد از Restart

اصل مهم داده

> **Database منبع اصلی حقیقت است.** تعداد مراحل، درصد پیشرفت، وضعیت‌ها و ترتیب باید از دادهٔ ذخیره‌شده بیاید؛ نه از عدد ثابت، localStorage یا HTML از پیش نوشته‌شده.

فایل‌های نمونه: **Sample2.zip** شامل AGENTS.md و مستندات roadmap-template، data-model و acceptance-checklist است.

جمع‌بندی

اگر بخواهیم همه‌چیز را در یک تصویر ببینیم



چهار نکته‌ای که باید به خاطر بماند

1. **AGENTS.md** قانون کلی پروژه است.
2. **SKILL.md** مسیر استفاده از یک توانایی تخصصی را نشان می‌دهد.
3. **Workflow** مسیر و ترتیب کار است؛ **Skill** توانایی داخل آن مسیر.
4. نمونهٔ خوب، ورودی و محدودیت را دقیق می‌گوید و بعد خروجی را با Build و تست بررسی می‌کند.

جملهٔ نهایی: Agent زمانی خروجی قابل‌اعتماد می‌دهد که بداند «قوانین چیست»، «چه توانایی‌ای لازم است»، «جزئیات کجا قرار دارد» و «نتیجه را چگونه بررسی کند».

پرسش و پاسخ